

Evaluating models honestly

A score is an estimate from a finite sample of rows. These habits make it a number that still holds on data the model has never seen.

THE QUESTION

How do you trust a number like "92% accuracy", and know it will hold on new data?

THE INTUITION

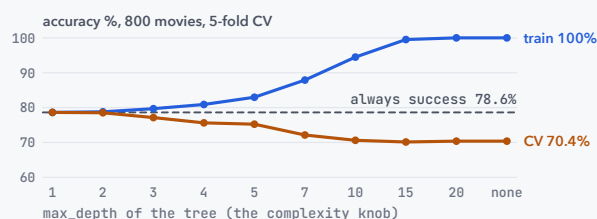
You want the average loss on new draws from the same distribution, and you can only estimate it on rows the model never saw. A row that informs any choice, from fitting a scaler to picking a depth, is no longer unseen.

KEY TAKEAWAY

Beat a dumb baseline, report mean $\pm$ std over K folds, fit every step inside the fold, split by time or group when rows depend on each other, and open the test set once.

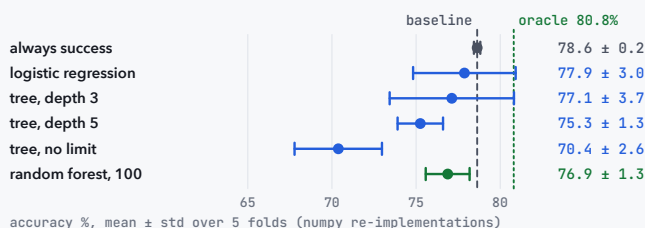
baseline $\rightarrow$ split $\rightarrow$ pipeline per fold $\rightarrow$ K-fold CV $\rightarrow$ mean $\pm$ std $\rightarrow$ pick, refit on all $\rightarrow$ test once

1 Training accuracy measures memorization



An unlimited tree gets 100% on its own rows and 70.4% on unseen ones. Held-out rows estimate the loss on new data: $\hat{E}_{\text{test}} = \frac{1}{n_{\text{test}}} \sum_i \mathcal{L}(f(x_i), y_i)$. A train and test gap above 10 points suggests overfitting.

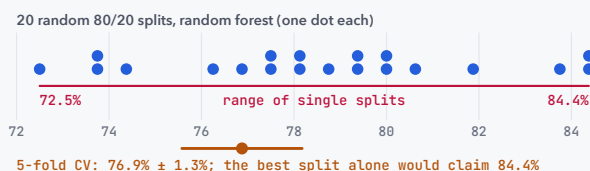
2 Start with a baseline the model must beat



```
from sklearn.dummy import DummyClassifier
base = DummyClassifier(
    strategy="most_frequent")
cross_val_score(base, X, y, cv=5).mean()
```

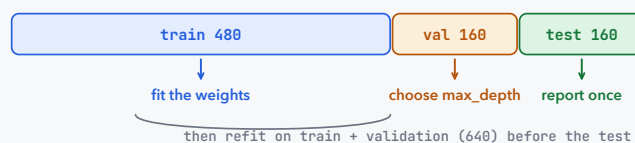
In the lab's 800 movies 78.6% succeed, and no model beats that on accuracy; an oracle that knows each true success probability gets 80.8%. Regression's baseline is the mean ($R^2 = 0$).

3 One split is one noisy draw



Which 160 movies land in the test set moves the same forest's score by 12 points. Report "76.9% $\pm$ 1.3% (5-fold CV)". Reporting the best of many seeds is cherry-picking; Dodge et al. (2020) showed seeds alone can reorder models.

4 Train, validation and test do different jobs



```
X_trval, X_test, y_trval, y_test = \
    train_test_split(X, y, test_size=0.2,
                    random_state=42)
X_tr, X_val, y_tr, y_val = \
    train_test_split(X_trval, y_trval,
                    test_size=0.25, random_state=42)
```

0.25 of 80% is 20%. Choosing among 100 depths by their test scores reports the luckiest one, so the test set stays sealed until the end.

5 K-fold validates every row exactly once

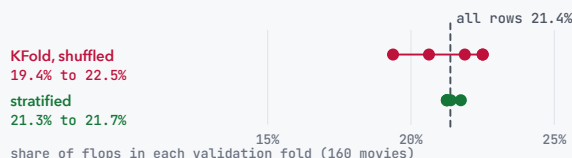
round 1	validate	train	train	train	train	acc 0.770
round 2	train	validate	train	train	train	acc 0.775
round 3	train	train	validate	train	train	acc 0.744
round 4	train	train	train	validate	train	acc 0.781
round 5	train	train	train	train	validate	acc 0.774

each movie is validated once, trained on 4 times · mean 0.769 $\pm$ 0.013

```
scores = cross_val_score(model, X, y, cv=5)
print(scores.mean(), scores.std())
```

K = 5 trains on 80% of the rows (a three-way split trains on 60%); K = 10 trains on 90% for twice the fits; leave-one-out (K = N) suits tiny datasets.

6 Stratify so each fold keeps the class ratio



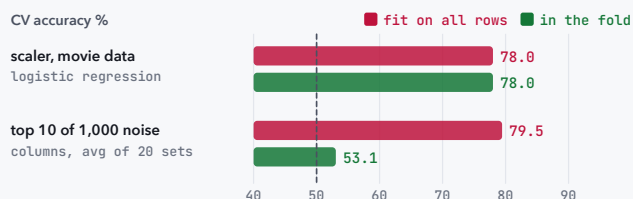
```
skf = StratifiedKFold(n_splits=5,
                      shuffle=True, random_state=42)
cross_val_score(model, X, y, cv=skf)
```

Stratifying deals each class round-robin into the folds. With 1% fraud, a plain fold could hold none. cv=5 on a classifier already stratifies, without shuffling.

Keep the validation rows unseen

Fit steps inside the fold, split along time or groups, read learning curves, pick the metric from the decision.

7 Fit preprocessing inside the fold

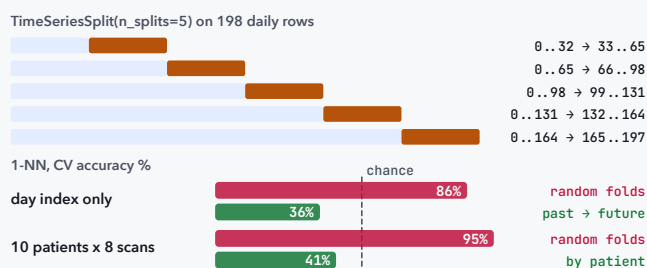


```
# leaky: the scaler saw every fold
Xs = StandardScaler().fit_transform(X)
cross_val_score(LogisticRegression(),
                 Xs, y, cv=5)

# right: refit on each training fold
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())])
cross_val_score(pipe, X, y, cv=5)
```

The size of a leak depends on the step. The scaler leak changed nothing on the movies, while picking 10 of 1,000 noise columns on all rows turned coin flips into 79.5%. Imputers and feature selectors go inside the pipeline too.

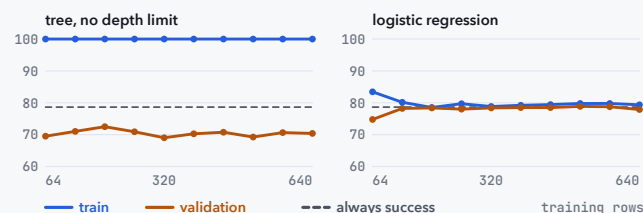
8 Split by time or group when rows depend



Random folds let a model peek: at tomorrow through today's neighbours, or at a patient through their other scans.

classification	StratifiedKfold
time series	TimeSeriesSplit
patient, speaker, document	GroupKfold (groups=ids)
one subject out	LeaveOneGroupOut
under 50 rows	LeaveOneOut
anything else	Kfold

9 Learning curves show if more rows help



```
learning_curve(model, X, y, cv=5,
               train_sizes=np.linspace(0.1, 1.0, 10))
```

A gap that narrows as rows grow (high variance): more data helps. Both flat and low (high bias): the model or features are too weak. Logistic regression is flat from 200 rows on; it needs better features.

10 Choose the metric from the decision

	always success	logistic reg.	tree, no limit
accuracy	0.786	0.779	0.784
balanced accuracy	0.500	0.551	0.580
recall on flops	0.000	0.152	0.363
ROC AUC	0.500	0.760	0.580

out-of-fold predictions, 800 movies; green marks the winner on each row

Accuracy rewards always saying success, ROC AUC rewards ranking movies by risk, and flop recall rewards catching flops (the tree flags 190 for 171 real ones). Set it with `cross_val_score(..., scoring="roc_auc")`.

11 Compare with CV, then refit the winner

```
cands = {"dummy": DummyClassifier(),
         "logreg": LogisticRegression(),
         "forest": RandomForestClassifier()}
for name, mdl in cands.items():
    s = cross_val_score(mdl, X, y, cv=5)
    print(name, s.mean(), s.std())
best = cands["logreg"].fit(X, y) # all rows
```

A search's best_score_ is optimistic; nested CV runs the search inside each outer fold (next: the tuning and experiment tracking sheet).

12 Pause and think

99% train, 60% test? Overfitting. Simplify, regularize or add data.

Several scans per patient? Use GroupKfold on patient id, or the model learns the patient.

Pick the depth on the test set? Its score is then the best of many tries, biased upward.

Predicting prices? Never shuffle; train on the past with TimeSeriesSplit.

89.5% CV when 89.4% of rows pass? It ties "always pass" (the notebook's student data, quoted). Scaled before CV? Possibly optimistic. A Pipeline removes the question.

baseline first · K-fold mean ± std · fit inside the fold · split by time or group · metric from the decision · test once

Source: lecture "Model evaluation", its evaluation notebook and the labs "From models to experiments" and "Cross-validation: sklearn and from scratch", nipunbatra.github.io/stt-ai-teaching. Movie data from the lab's generator (800 movies, seed 42). Numbers come from numpy re-implementations of the lab's models (5-fold stratified CV) that match the notebooks' saved outputs; scikit-learn may differ by a point. Time-series, patient and noise-column demos are illustrative. The maths of cross-validation is on the machine learning cross-validation sheet.